Listing 7.15. Implementing push() for a lock-free queue with a reference-counted tail

Listing 7.16. Popping a node from a lock-free queue with a reference-counted tail

Listing 7.17. Releasing a node reference in a lock-free queue

Listing 7.18. Obtaining a new reference to a node in a lock-free queue

Listing 7.19. Freeing an external counter to a node in a lock-free queue

Listing 7.20. pop() modified to allow helping on the push() side

Listing 7.21. A sample push() with helping for a lock-free queue

Chapter 8. Designing concurrent code

Listing 8.1. Parallel Quicksort using a stack of pending chunks to sort

Listing 8.2. A naïve parallel version of std::accumulate (from listing 2.8)

Listing 8.3. A parallel version of std::accumulate using std::packaged_task

Listing 8.4. An exception-safe parallel version of std::accumulate

Listing 8.5. An exception-safe parallel version of std::accumulate using std::async

Listing 8.6. Separating GUI thread from task thread

Listing 8.7. A parallel version of std::for_each

Listing 8.8. A parallel version of std::for_each using std::async

Listing 8.9. An implementation of a parallel find algorithm

Listing 8.10. An implementation of a parallel find algorithm using std::async

Listing 8.11. Calculating partial sums in parallel by dividing the problem

Listing 8.12. A simple barrier class

Listing 8.13. A parallel implementation of partial_sum by pairwise updates